

Universidad del Valle de Guatemala
Facultad de Ingenieria
Departamento de Ciencias de la Computacion
Computacion Paralela y Distribuida - Seccion 20

Proyecto #1

Screensaver de fluidos paralelizado con OpenMP
Simulacion de Navier-Stokes (metodo Stable Fluids) con n-cuerpos

Integrantes

TonitoMC
Javier-Espana
AngelEsquit

Guatemala, septiembre de 2026

Índice

1. Introduccion	2
2. Antecedentes	2
3. Descripcion del problema	2
4. Metodologia: PCAM	3
5. Version secuencial	3
6. Paralelizacion iterativa	3
6.1. Paso 1: Barnes-Hut para n-body	3
6.2. Paso 2: paralelizar los loops independientes	4
6.3. Paso 3: red-black Gauss-Seidel	4
6.4. Paso 4: ajuste de <code>schedule</code>	4
6.5. Paso 5: ajuste de <code>collapse(2)</code> (cancelado)	4
6.6. Paso 6: flags de compilacion	4
7. Mecanismos de sincronizacion	5
8. Resultados	5
8.1. Tabla maestra	5
8.2. Escalabilidad por hilos	5
8.3. Barrido de <code>schedule/chunk</code>	6
8.4. Flags de build	6
9. Conclusiones	6
10.Recomendaciones	7
A. Anexo 1: Diagrama de flujo del programa	9
B. Anexo 2: Catalogo de funciones	12
C. Anexo 3: Bitacora de pruebas	16

1. Introduccion

Este informe documenta el desarrollo de un screensaver de fluidos, paralelizado con OpenMP. El programa simula un fluido incompresible en 2D usando el metodo Stable Fluids de Jos Stam, con fuentes de tinta que se mueven por gravedad mutua (n-cuerpos).

El objetivo del proyecto es tomar una version secuencial funcional y mejorarla de forma iterativa, midiendo speedup y eficiencia en cada paso. Cada paso agrega una sola mejora (algoritmica o de paralelizacion) para que su impacto se pueda medir por separado.

2. Antecedentes

OpenMP es una API para programacion paralela de memoria compartida en C/C++/Fortran. Funciona mediante directivas de compilador (`#pragma omp`) que le indican al compilador que partes del codigo se pueden repartir entre varios hilos. A diferencia de MPI (memoria distribuida), todos los hilos de OpenMP comparten el mismo espacio de memoria, lo cual simplifica la programacion pero exige cuidado con condiciones de carrera.

El problema elegido, simulacion de fluidos, es un caso clasico de paralelizacion de memoria compartida: la malla se puede dividir en regiones, y (con el cuidado correcto de dependencias) cada region se puede actualizar en un hilo distinto. El metodo de Stam (*Stable Fluids*) resuelve las ecuaciones de Navier-Stokes con pasos incondicionalmente estables, lo que permite pasos de tiempo grandes sin que la simulacion explote numericamente [1].

Para el movimiento de las fuentes de tinta se implemento un sistema de n-cuerpos con gravedad mutua, acelerado con un quadtree Barnes-Hut [2], que reduce el costo de $O(n^2)$ a $O(n \log n)$.

3. Descripcion del problema

El programa dibuja un lienzo de al menos 640x480 px donde varias fuentes de tinta de colores inyectan color y momentum en un campo de velocidad. El campo de velocidad se resuelve cada frame (difusion, adveccion, proyeccion de Hodge) y arrastra la tinta, generando remolinos. Las fuentes mismas se mueven por atraccion gravitacional mutua y rebotan en los bordes de la malla.

Parametros configurables por linea de comandos: resolucion de la malla (`-n`), cantidad de fuentes (`-f`), tamano de ventana (`-W/-H`), semilla aleatoria (`-s`), viscosidad (`-v`), difusion (`-d`) y pantalla completa (`-p`). Los valores por defecto y rangos validos estan en la Tabla 1.

Flag	Rango	Default
<code>-n</code>	16 – 1024	256
<code>-f</code>	1 – 256	6
<code>-W</code>	min. 640	1920
<code>-H</code>	min. 480	1080
<code>-v</code>	0.0 – 1.0	0.0
<code>-d</code>	0.0 – 1.0	0.0001

Cuadro 1: Parametros de linea de comandos.

4. Metodologia: PCAM

Se aplico el metodo PCAM (Particionamiento, Comunicacion, Aglomeracion, Mapeo) para decidir que y como paralelizar:

Particionamiento La malla de la simulacion se piensa como un conjunto de celdas independientes. Cada operacion por celda (inyeccion de fuente, disipacion, adveccion, proyeccion, render) es, en principio, una tarea separada.

Comunicacion La mayoria de las operaciones solo leen el estado del frame anterior y escriben el nuevo, sin comunicacion entre celdas dentro del mismo paso. La excepcion es el solver de presion/difusion (Gauss-Seidel), que en su forma original lee valores ya actualizados de celdas vecinas en la misma iteracion, creando una dependencia estricta que impide paralelizar directamente.

Agglomeracion En vez de repartir celda por celda (overhead de sincronizacion altisimo), se agrupan filas en bloques (*chunks*) que cada hilo procesa de corrido.

Mapeo Los bloques se reparten entre los hilos disponibles via `#pragma omp parallel for`, con la politica de reparto (`schedule`) elegida segun el tamano de la malla (ver Seccion 6.4).

5. Version secuencial

La version secuencial (rama `sequential`) sirve de linea base fija para medir speedup: una vez medida, no se vuelve a tocar, ni un commit mas, para que el punto de comparacion no se mueva mientras se paraleliza.

Usa Gauss-Seidel fila por fila para el solver de presion/difusion y fuerza bruta $O(f^2)$ para la gravedad del n-body (sin quadtree, cada fuente contra cada otra fuente). El resto de la estructura del programa (parseo de argumentos, asignacion de memoria, bucle principal, limpieza al salir) es identica a las versiones paralelas — los pasos del roadmap solo tocan el algoritmo del solver y el n-body, no la estructura general.

Antes de arrancar cualquier paralelizacion, esta version se probó: compila limpio con `-Wall -Wextra -O2 -std=c11`, corre sin crashear para el rango completo de `-n` y `-f`, y `valgrind --leak-check=full` reporta 0 bytes perdidos y 0 errores (la limpieza al salir libera los 12 campos de la simulacion, las fuentes y las estrellas de fondo, ver `cleanup`: en `main.c`). Tambien se probó el parseo de argumentos con entradas invalidas (no numericas, fuera de rango, valores faltantes, flags desconocidos) — ver Anexo 2 para el detalle de `read_int/read_float`.

6. Paralelizacion iterativa

Siguiendo el espiritu de OpenMP como herramienta iterativa, la paralelizacion se hizo en pasos, cada uno en su propia rama, midiendo la misma bateria de pruebas despues de cada uno.

6.1. Paso 1: Barnes-Hut para n-body

Se reemplazo el calculo de gravedad de fuerza bruta ($O(f^2)$) por un quadtree Barnes-Hut ($O(f \log f)$). Este paso es puramente algoritmico, sin OpenMP todavia. A las resoluciones de malla probadas, el solver de fluidos domina tanto el costo que el cambio no se nota en el FPS total (el n-body es una fraccion pequena del trabajo por frame).

6.2. Paso 2: paralelizar los loops independientes

Se agrego `#pragma omp parallel for` a los sitios donde cada celda de salida es independiente: inyeccion de fuentes, disipacion de tinta, adveccion, proyeccion de Hodge y render. El solver de presion/difusion seguia secuencial. Con 20 hilos, este paso ya da un salto importante de FPS (Tabla 2), pero el techo sigue siendo el solver.

6.3. Paso 3: red-black Gauss-Seidel

El cambio mas grande de toda la cadena. El solver de Gauss-Seidel original lee y escribe la misma malla en el mismo barrido, celda por celda en orden, lo que crea una dependencia secuencial estricta. La solucion es un recorrido *red-black*: se divide la malla en un tablero de ajedrez por paridad $(i + j) \bmod 2$. Los 4 vecinos de una celda de un color son siempre del color opuesto, entonces todas las celdas del mismo color se pueden actualizar en paralelo sin pisarse entre si. Cada iteracion pasa a ser: actualizar todas las rojas (barrera implicita), actualizar todas las negras (barrera implicita).

Este cambio finalmente paraleliza el hotspot principal del programa. El resultado es el salto de rendimiento mas grande del proyecto: a la configuracion default ($n = 256$, $f = 6$), el FPS pasa de 25.20 (Paso 2) a 88.27 (Paso 3), un $3.5\times$ adicional.

6.4. Paso 4: ajuste de schedule

Con el hotspot ya paralelo, se probo si el `schedule(dynamic, 16)` usado en todo el codigo era la mejor eleccion. Se cambiaron temporalmente los `#pragma` a `schedule(runtime)` (que se puede configurar sin recompilar via la variable de entorno `OMP_SCHEDULE`) y se barrio `static`, `dynamic` y `guided` con distintos tamanos de chunk, en malla chica y malla grande (Tabla 5).

El resultado principal: el `dynamic, 16` usado en todo el codigo es buena eleccion en malla chica, pero es la peor opcion razonable en malla grande. `static` (con el chunk por defecto del compilador) le saca $2.5\times$ a `dynamic, 16` en $n = 700$ (54.47 vs. 21.93 FPS).

Con este resultado, se aplico `schedule(static)` de verdad al codigo de la rama `04-schedule-tuning`, reemplazando el `schedule(dynamic, 16)` hardcodedo en todos los sitios del catalogo. Al remedir la bateria estandar con el cambio real (no solo con `OMP_SCHEDULE`), el resultado es un **tradeoff**, no una mejora universal: en malla chica `static` pierde un poco contra el `dynamic, 16` anterior (B6: 88.27 $\rightarrow$ 80.68 FPS, $\sim -8.6\%$), pero en malla grande gana fuerte (I6: 28.32 $\rightarrow$ 53.47 FPS, $\sim +89\%$; C6: 45.79 $\rightarrow$ 68.57 FPS, $\sim +50\%$). Ver Tabla 2 para los numeros completos.

6.5. Paso 5: ajuste de collapse(2) (cancelado)

Se planeo comparar `collapse(2)` contra paralelizar solo el loop externo, para ver el efecto en mallas chicas (menos filas que nucleos disponibles). Este paso se cancelo antes de obtener numeros confiables y no se incluye en los resultados de este informe.

6.6. Paso 6: flags de compilacion

Se probo cambiar el flag de optimizacion del compilador de `-O2` (usado hasta ahora) a `-O3`, y adicionalmente `-march=native` para autovectorizacion especifica del CPU de la maquina de pruebas. Los resultados estan en la Tabla 6.

`-O3` da una ganancia real de $\sim 11\%$ en malla grande sobre `-O2`, sin costo en malla chica. `-march=native` no aporta nada adicional sobre `-O3` solo, lo que sugiere que los loops calientes no se benefician de registros vectoriales mas anchos en este CPU, o que el set de instrucciones base que asume GCC ya alcanza.

7. Mecanismos de sincronizacion

- **#pragma omp parallel for**: la forma principal de paralelismo en el proyecto. Reparte las iteraciones de un loop **for** entre los hilos disponibles, con una barrera implicita al final (todos los hilos esperan a que termine el ultimo antes de seguir).
- **#pragma omp parallel + #pragma omp for**: usado en el solver (Seccion 6.3) para abrir una sola region paralela que envuelve las 20 iteraciones del Gauss-Seidel red-black, en vez de abrir/cerrar el team de hilos 40 veces por frame (costo fijo de creacion de hilos evitado).
- **#pragma omp single**: dentro de la region paralela del solver, aplica la condicion de frontera con un solo hilo mientras el resto espera en la barrera implicita del **single**, evitando que varios hilos escriban la misma celda de borde a la vez.
- **schedule(dynamic, chunk)**: politica de reparto de trabajo por defecto usada en todo el codigo. Reparte bloques de **chunk** iteraciones bajo demanda: un hilo que termina su bloque pide el siguiente en vez de quedarse ocioso esperando a los demas. La Seccion 6.4 muestra que no siempre es la mejor opcion.
- **collapse(2)**: en los loops anidados de adveccion, proyeccion y render, funde las dos dimensiones (fila, columna) en un solo rango repartible. Sin esto, una malla mas angosta que la cantidad de hilos dejaria nucleos sin trabajo.
- **Variables privadas (private(...))**: cada hilo necesita su propia copia de las variables escalares temporales del cuerpo del loop (por ejemplo, coordenadas de origen interpoladas en adveccion). Sin esto, los hilos pisarian el mismo valor compartido y corromperian el resultado.

No se usaron mutex ni secciones criticas explicitas: el diseno evita condiciones de carrera por construccion (cada celda de salida se escribe una sola vez, por un solo hilo, en cada paso), asi que las barreras implicitas de **omp for** y el **single** para la frontera son suficientes.

8. Resultados

8.1. Tabla maestra

La Tabla 2 muestra el FPS promedio por configuracion (malla $\times$ fuentes), en cada paso de la cadena, con 20 hilos donde aplica. Metodologia: cada fila se corrio con semilla fija (**-s 42**), se promedia el FPS ignorando las primeras lineas de calentamiento.

En la configuracion default ($n = 256$, $f = 6$), el FPS pasa de 10.52 (secuencial) a 88.27 (Paso 3), un speedup acumulado de $\sim 8,4\times$. El Paso 4 (**static**) es un tradeoff frente a Paso 3: pierde un poco en malla chica pero gana fuerte en malla grande (hasta $\sim +89\%$ en $n=700$, Tabla 2).

8.2. Escalabilidad por hilos

La Tabla 3 muestra el FPS variando **OMP_NUM_THREADS** en la fila B ($n=256$, $f=6$). Antes del Paso 3 (solver todavia secuencial), el escalado se estanca alrededor de $2,6\times$ (techo de Amdahl: el solver secuencial limita cuanto puede acelerar el resto). Desde el Paso 3 en adelante, con el solver ya paralelo, el escalado sube a $\sim 6\times$.

Tambien se corrio la misma curva en una malla grande ($n=700$, $f=12$), comparando el **schedule** actual contra **static** (el mejor encontrado para malla grande, Tabla 4). Con **static**, 20 hilos da

Fila	sequential	01-rb-tree	02-omp-loops	03-omp-solver (dynamic,16)	04-schedule (static)
A6 (n=64, f=6)	17.21	17.19	118.40	88.04	83.87
A12 (n=64, f=12)	17.16	17.20	117.43	88.31	84.39
B6 (n=256, f=6)	10.52	10.51	25.20	88.27	80.68
B12 (n=256, f=12)	10.95	10.95	25.24	88.66	80.92
C6 (n=512, f=6)	3.75	3.76	6.72	45.79	68.57
C12 (n=512, f=12)	4.30	4.30	6.89	52.33	69.53
H6 (n=600, f=6)	2.69	2.69	4.46	33.72	61.41
H12 (n=600, f=12)	3.09	3.09	4.79	41.39	62.29
I6 (n=700, f=6)	1.90	1.91	3.00	28.32	53.47
I12 (n=700, f=12)	2.10	2.10	3.21	35.57	54.76
D6 (n=1024, f=6)	0.92	0.92	1.15	5.10	4.64
D12 (n=1024, f=12)	1.00	0.99	1.23	6.32	3.21
E (n=256, f=4)	7.52	7.52	24.39	87.09	n/m
F (n=256, f=64)	11.39	11.39	25.36	87.28	n/m
G (n=256, f=256)	11.37	11.38	25.30	85.07	n/m

Cuadro 2: FPS promedio por paso de la cadena de paralelizacion. n/m = no medido con el schedule nuevo todavia.

Hilos	FPS 02-omp-loops	FPS 03-omp-solver	FPS 04-schedule
1	9.95	14.72	14.72
2	14.94	25.39	25.52
4	20.33	45.94	45.91
8	23.57	67.84	68.27
16	25.52	83.27	83.66
20	25.57	87.76	87.72

Cuadro 3: FPS vs. cantidad de hilos, fila B (n=256, f=6).

48.11 FPS contra los 34.59 FPS del **dynamic**, 16 actual en la misma fila: +39% solo cambiando el schedule.

Con **static** el pico esta en 16 hilos, no en 20: a 700 filas, el reparto estatico en 20 partes no cae tan parejo como en 16.

8.3. Barrido de schedule/chunk

8.4. Flags de build

9. Conclusiones

- Paralelizar el solver de presion/difusion (red-black Gauss-Seidel) fue, por lejos, el cambio de mayor impacto: FPS default subio de 25.20 a 88.27 (3,5×), y la escalabilidad por hilos paso de $\sim 2,6\times$ a $\sim 6\times$.
- Antes de paralelizar el hotspot real, paralelizar el resto del programa (Paso 2) ayuda, pero el techo de Amdahl limita cuanto se puede ganar mientras el cuello de botella siga secuencial.
- La eleccion de **schedule** de OpenMP importa mas de lo esperado y no tiene un ganador

Hilos	FPS <code>static</code>	FPS <code>dynamic,16</code> (actual)
1	6.82	—
2	15.25	—
4	30.58	—
8	41.02	—
16	53.80	—
20	48.11	34.59

Cuadro 4: FPS vs. hilos en malla grande ($n=700$, $f=12$).

OMP_SCHEDULE	FPS $n=256$	FPS $n=700$
<code>static</code> (default)	83.40	54.47
<code>static,4</code>	83.57	43.01
<code>static,16</code>	84.53	46.87
<code>static,64</code>	72.89	44.42
<code>dynamic,1</code>	31.19	5.55
<code>dynamic,4</code>	64.36	15.77
<code>dynamic,16</code> (actual)	87.58	21.93
<code>dynamic,64</code>	76.88	2.02
<code>guided</code> (default)	96.80	49.42
<code>guided,16</code>	94.71	3.43

Cuadro 5: Barrido de `schedule/chunk`, 20 hilos, $f=6$.

universal: `dynamic, 16` era mejor en malla chica, `static` gana fuerte en malla grande ($2,5\times$ en el barrido, hasta $+89\%$ al aplicarlo de verdad al código). Se aplicó `static` a la rama `04-schedule-tuning`; el resultado real confirma el tradeoff: pierde $\sim 8\%$ en malla chica para ganar hasta $\sim 89\%$ en malla grande.

- Cambiar `-02` a `-03` da una ganancia gratuita de $\sim 11\%$ en malla grande sin tocar código. `-march=native` no aporta nada extra sobre `-03` en este CPU.
- Barnes-Hut para el n -body no se nota en el FPS a las resoluciones probadas porque el solver de fluidos domina el costo total; el algoritmo de n -body es una fracción pequeña del trabajo por frame.

10. Recomendaciones

- Ya se aplicó `static` en `04-schedule-tuning`. Si el uso real del programa favorece mallas chicas ($n \leq 256$), vale la pena reconsiderar, ya que ahí `dynamic, 16` sigue siendo mejor; para mallas grandes, `static` es la mejor elección medida. Una opción futura es dejarlo configurable vía `schedule(runtime)` y elegir en base a `-n` al arrancar.
- Cambiar `-02` a `-03` en el `Makefile`: es una ganancia gratuita sin riesgo.
- Retomar el Paso 5 (`collapse(2)`) con una metodología más controlada (por ejemplo, sin renderizado en pantalla completa) si se quiere ese número para mallas chicas con pocos núcleos.

Build	FPS n=256,f=6	FPS n=700,f=12
-O2 (actual)	83.74	32.18
-O3	83.16	35.72
-O3 -march=native	84.03	35.76

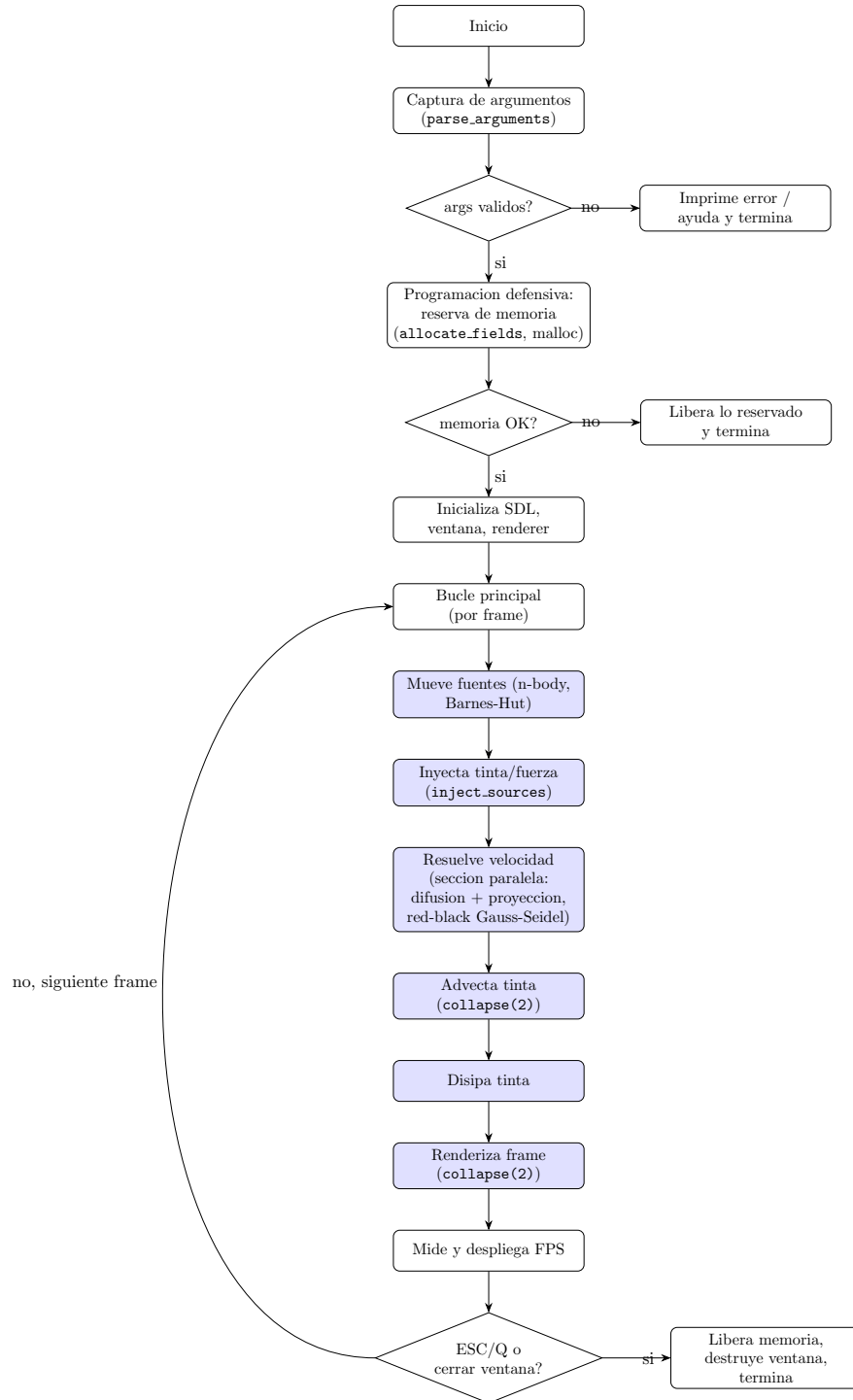
Cuadro 6: FPS por flag de optimizacion, 20 hilos.

- Para mallas mayores a $n = 1024$ o cantidades de fuentes mayores a 256, medir de nuevo: el comportamiento del schedule optimo podria cambiar fuera del rango probado.

Referencias

- [1] Stam, J. (1999). *Stable Fluids*. En Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99), 121-128.
- [2] Barnes, J., & Hut, P. (1986). *A hierarchical $O(N \log N)$ force-calculation algorithm*. Nature, 324(6096), 446-449.
- [3] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. <https://www.openmp.org/specifications/>
- [4] Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

A. Anexo 1: Diagrama de flujo del programa



Los bloques en azul son las secciones paralelizadas con OpenMP. La sincronizacion entre ellas es implicita: cada `#pragma omp for` tiene una barrera al final, y la seccion de velocidad usa `#pragma omp single` para la condicion de frontera dentro de su propia region paralela (ver Seccion 6.3).

Que pasa dentro de cada bloque, un frame a la vez

El diagrama de arriba muestra el orden, pero no el detalle de cada bloque. Esta seccion entra a ese detalle, bloque por bloque, en el orden real de ejecucion dentro de `main.c`.

1. Mueve fuentes (n-body, Barnes-Hut) — `update_nbody_sources`. Reconstruye un quadtree desde cero cada frame (las fuentes se mueven, no tiene sentido mantenerlo entre frames). Primero inserta las f fuentes una por una (`bh_insert`, recursivo, subdivide un nodo hoja en 4 cuando cae una segunda fuente adentro). Despues, para cada fuente, recorre el arbol (`bh_accumulate`): si un nodo esta lo bastante lejos en relacion a su tamano ($\theta = 0,5$), se trata como un solo punto de masa en vez de bajar a sus 4 hijos — eso es lo que baja el costo de $O(f^2)$ a $O(f \log f)$. Con la aceleracion acumulada, integra velocidad y posicion, y aplica un rebote amortiguado si una fuente llegaria a salir del area valida de inyeccion. Esta etapa sigue secuencial en toda la cadena: el arbol se construye una vez por frame y depende del resultado del frame anterior en cada insercion, no vale la pena paralelizarla a la cantidad de fuentes que se usan tipicamente ($f \leq 256$).

2. Inyecta tinta/fuerza — `inject_sources`. Primero limpia los buffers "previous" de velocidad y tinta (van a recibirse de nuevo este frame). Para cada fuente, calcula dos gaussianas centradas en su posicion (continua, no la celda entera): una ancha que da el color/empuje general, y una angosta (core) que agrega un nucleo brillante casi blanco en el centro. La direccion del chorro rota con $\cos(\text{phase})/\sin(\text{phase})$, generando el efecto espiral. No esta paralelizada: el radio de inyeccion por fuente es chico (unas pocas celdas) y varias fuentes pueden solapar en la misma celda si estan cerca, lo que forzaria una seccion critica o reduccion — no vale la pena al tamano tipico de `-f`.

3. Resuelve velocidad — `velocity_step`. El bloque mas caro del frame, y donde vive el paralelismo real. Es una secuencia fija: fuerza $\rightarrow$ difusion $\rightarrow$ proyeccion $\rightarrow$ autoadveccion $\rightarrow$ proyeccion otra vez (difundir y autoadveccion cada uno reintroduce algo de divergencia, por eso se proyecta dos veces). La difusion y cada proyeccion llaman a `solve_linear`, que abre una sola region `#pragma omp parallel` y adentro corre las 20 iteraciones de Gauss-Seidel red-black: 20 veces (actualizar celdas rojas, barrera implicita, actualizar celdas negras, barrera implicita). Abrir la region paralela una sola vez en vez de 40 veces (20 iteraciones $\times$ 2 colores) evita el costo fijo de crear el team de hilos repetidamente, que en mallas chicas se nota. La condicion de frontera se aplica con `#pragma omp single` dentro de esa misma region: un solo hilo la escribe mientras los demas esperan en la barrera implicita del `single`, sin salir de la region paralela.

4. Advecta tinta — `ink_step` (una vez por canal R, G, B). Adveccion semi-Lagrangiana: para cada celda destino, calcula de donde vino ese fluido dando un paso hacia atras a lo largo de su propio campo de velocidad, y agarra el valor de ahi por interpolacion bilineal de las 4 celdas reales alrededor de ese punto (fraccional). Cada celda destino solo lee el campo anterior, nunca lo escribe, asi que es paralelizable sin ningun cuidado extra. Usa `#pragma omp parallel for collapse(2)`: funde el loop de `i` y el de `j` en un solo rango repartible, para que una malla angosta (menos filas que hilos) siga repartiendo trabajo entre todos. Se llama 3 veces (una por canal de color), cada una independiente de las otras.

5. Disipa tinta — `dissipate_ink`. Multiplica cada celda de cada canal por `DISSIPATION` (0.995), sin eso la tinta se acumularia hasta saturar la pantalla a blanco. Loop plano sobre `celdas_totales`, cada celda independiente, `#pragma omp parallel for`.

6. Renderiza frame — `render_ink`. Recorre cada pixel de la textura (no cada celda de la malla — la resolución de ventana suele ser mayor que `-n`), interpola bilinealmente la densidad de tinta en esa posición, aplica un mapeo de tono estilo Reinhard y un ajuste de saturación, y calcula el canal alfa a partir del brillo máximo de los 3 canales (para que el fondo se vea donde no hay tinta). También usa `collapse(2)`: sin él, con una ventana más angosta que alta (o viceversa) los hilos no se repartirían parejo solo sobre el loop externo.

7. Mide FPS y actualiza pantalla. Cuenta frames acumulados y, cada ~ 0.5 s de tiempo real, calcula el promedio, lo imprime a `stdout` y lo pone en el título de la ventana. No es parte del cálculo de la simulación, es solo instrumentación.

B. Anexo 2: Catalogo de funciones

config.c

Funcion	Entradas	Salidas	Descripcion
<code>parse_arguments</code>	<code>argc</code> (int), <code>argv</code> (char**)	<code>config</code> (Config*, por referencia), retorna int (1 ok, 0 error, -1 ayuda)	Recorre <code>argv</code> y llena <code>config</code> con los valores validados.
<code>read_int</code>	<code>text</code> (char*), rango min/max	<code>out</code> (int*), retorna 1/0	Convierte texto a int validando formato y rango.
<code>read_float</code>	<code>text</code> (char*), rango min/max	<code>out</code> (float*), retorna 1/0	Igual que <code>read_int</code> pero para float.
<code>print_help</code>	<code>program_name</code> (char*)	ninguna (imprime a stdout)	Imprime la pantalla de uso del programa.

fields.c

Funcion	Entradas	Salidas	Descripcion
<code>allocate_fields</code>	<code>fields</code> (FluidFields*), <code>resolution</code> (int)	retorna int (1 ok, 0 error)	Reserva memoria para los 12 campos de la simulacion (velocidad, tinta, presion, divergencia).
<code>free_fields</code>	<code>fields</code> (FluidFields*)	ninguna	Libera los 12 campos y pone los punteros en NULL.
<code>clear_fields</code>	<code>fields</code> (FluidFields*)	ninguna	Pone todos los campos en cero (usado al reiniciar con la tecla R).

sources.c

Funcion	Entradas	Salidas	Descripcion
<code>init_sources</code>	<code>sources</code> (InkSource*), <code>count</code> , <code>resolution</code> (int)	ninguna	Inicializa posicion, velocidad, masa, color y fuerza de cada fuente.

<code>inject_sources</code>	<code>sources</code> (InkSource*), <code>count</code> (int), <code>fields</code> (FluidFields*), <code>aspect</code> (float)	ninguna (modifica <code>fields</code>)	Deposita tinta y momentum de cada fuente en un vecindario gaussiano.
<code>dissipate_ink</code>	<code>fields</code> (FluidFields*)	ninguna	Reduce la tinta de cada celda por un factor fijo (paralelo con <code>omp parallel for</code>).

`solver.c`

Funcion	Entradas	Salidas	Descripcion
<code>apply_boundary</code>	<code>bnd_type</code> (int), <code>field</code> (float*)	ninguna	Aplica condiciones de frontera (rebote en paredes).
<code>add_source</code>	<code>dest</code> , <code>source</code> (float*), <code>dt</code> (float), <code>n</code> (int)	ninguna	Suma el termino fuente a cada celda (paralelo).
<code>relax_color</code>	<code>field</code> , <code>field_prev</code> (float*), <code>a</code> , <code>inv_c</code> (float), <code>parity</code> (int)	ninguna	Actualiza las celdas de una paridad del tablero red-black (paralelo, dentro de region <code>omp parallel</code>).
<code>solve_linear</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>a</code> , <code>c</code> (float)	ninguna	Corre las 20 iteraciones de Gauss-Seidel red-black dentro de una sola region paralela.
<code>diffuse</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>diff</code> , <code>dt</code> (float), <code>n</code> (int)	ninguna	Arma los coeficientes y llama a <code>solve_linear</code> para difusion implicita.
<code>advect</code>	<code>bnd_type</code> (int), <code>field</code> , <code>field_prev</code> (float*), <code>vel_x</code> , <code>vel_y</code> (float*), <code>dt</code> (float)	ninguna	Adveccion semi-Lagrangiana (paralelo, <code>collapse(2)</code>).
<code>project</code>	<code>vel_x</code> , <code>vel_y</code> , <code>pressure</code> , <code>divergence</code> (float*)	ninguna	Proyeccion de Hodge: mide divergencia, resuelve presion, resta el gradiente de la velocidad.
<code>swap_fields</code>	<code>field_a</code> , <code>field_b</code> (float**)	ninguna (intercambia punteros)	Intercambia los punteros de dos buffers de campo.

<code>ink_step</code>	<code>ink, ink_prev (float**), vel_x, vel_y (float*), diff, dt (float), n (int)</code>	ninguna	Un paso completo (difusion + adveccion) para un canal de tinta.
<code>velocity_step</code>	<code>fields (FluidFields*), viscosity, dt (float)</code>	ninguna	Un paso completo de velocidad: fuerza, difusion, proyeccion, autoadveccion, proyeccion.

`nbody.c`

Funcion	Entradas	Salidas	Descripcion
<code>bh_new_node</code>	<code>cx, cy, half (float)</code>	retorna int (indice del nodo)	Crea un nodo nuevo del quadtree Barnes-Hut.
<code>bh_quadrant</code>	<code>n (QuadNode*), x, y (float)</code>	retorna int (0-3)	Determina en que cuadrante cae un punto.
<code>bh_subdivide</code>	<code>idx (int)</code>	ninguna	Crea los 4 hijos de un nodo.
<code>bh_insert</code>	<code>idx, body (int), sources (InkSource*), depth (int)</code>	ninguna	Inserta una fuente en el arbol recursivamente.
<code>bh_accumulate</code>	<code>idx, body (int), sources (InkSource*), acc_x, acc_y (float*)</code>	ninguna (acumula en <code>acc_x/acc_y</code>)	Recorre el arbol acumulando aceleracion gravitacional sobre una fuente.
<code>update_nbody_sources</code>	<code>sources (InkSource*), count, resolution (int)</code>	ninguna	Reconstruye el arbol, calcula gravedad, integra posicion/velocidad y aplica rebote en bordes.

`render.c`

Funcion	Entradas	Salidas	Descripcion
<code>sample_bilinear</code>	<code>field (float*), fx, fy (float)</code>	retorna float	Interpola bilinealmente un campo en una posicion continua.

<code>render_ink</code>	<code>texture</code> (<code>SDL_Texture*</code>), <code>fields</code> (<code>FluidFields*</code>), <code>width</code> , <code>height</code> (int)	ninguna	Vuelca la densidad de tinta a la textura SDL, un pixel a la vez (paralelo, <code>collapse(2)</code>).
-------------------------	--	---------	--

`background.c`

Funcion	Entradas	Salidas	Descripcion
<code>init_background</code>	<code>stars</code> (<code>Star*</code>), <code>count</code> , <code>width</code> , <code>height</code> (int)	ninguna	Coloca estrellas de fondo en posiciones aleatorias.
<code>update_background</code>	<code>stars</code> (<code>Star*</code>), <code>count</code> (int)	ninguna	Avanza el parpadeo de cada estrella.
<code>draw_background</code>	<code>renderer</code> (<code>SDL_Renderer*</code>), <code>stars</code> (<code>Star*</code>), <code>count</code> (int)	ninguna	Dibuja las estrellas de fondo.

`utils.c`

Funcion	Entradas	Salidas	Descripcion
<code>random_range</code>	<code>min</code> , <code>max</code> (float)	retorna float	Numero pseudoaleatorio uniforme en $[min, max]$.
<code>clamp</code>	<code>value</code> , <code>min</code> , <code>max</code> (float)	retorna float	Limita un valor al rango $[min, max]$.

`main.c`

Funcion	Entradas	Salidas	Descripcion
<code>main</code>	<code>argc</code> (int), <code>argv</code> (<code>char**</code>)	retorna int (codigo de salida)	Punto de entrada: parsea argumentos, inicializa SDL y memoria, corre el bucle principal, libera todo al salir.

C. Anexo 3: Bitacora de pruebas

Metodologia

Cada configuracion se corrio con semilla fija (`-s 42`), ignorando las primeras lineas de FPS (calentamiento/cache fria), y promediando el resto. La bateria estandar (15 configuraciones, Tabla 16) se repitio identica despues de cada paso de la cadena; son 15 mediciones por paso, mas de las 10 minimas requeridas.

Fila	-n	-f	Motivo
A6/A12	64	6/12	mallla chica, referencia rapida
B6/B12	256	6/12	configuracion default
C6/C12	512	6/12	mallla grande, domina el solver
H6/H12	600	6/12	rango de FPS medio observado
I6/I12	700	6/12	extremo superior de ese rango
D6/D12	1024	6/12	mallla muy grande, limite superior
E	256	4	pocas fuentes
F	256	64	muchas fuentes
G	256	256	limite superior de fuentes

Cuadro 16: Bateria estandar de pruebas (15 configuraciones).

Para el barrido de `schedule` (Tabla 5) se probaron 10 combinaciones de politica/chunk, cada una en 2 tamanos de mallla: 20 mediciones. Para la escalabilidad por hilos (Tablas 3 y 4) se probaron 6 cantidades de hilos por curva.

Captura de una corrida

A continuacion, salida real de consola de una corrida de referencia ($n = 256$, $f = 6$, semilla 42), mostrando el arranque, calentamiento y estabilizacion del FPS:

```
Fluid simulation (Navier-Stokes) - SEQUENTIAL version
Grid       : 256 x 256 cells (65536 interior cells)
Sources    : 6
Window     : 1920 x 1080 px
Seed       : 42
visc / diff : 0.00000 / 0.00010
FPS= 12.73
FPS= 11.41
FPS= 12.96
FPS= 13.80
FPS= 75.53
FPS= 83.46
FPS= 83.82
FPS= 82.31
FPS= 84.26
FPS= 85.41
FPS= 84.09
FPS= 63.32
FPS= 74.58
FPS= 70.27
FPS= 77.99
```

Nota: la etiqueta "SEQUENTIAL version.^{en} el encabezado es texto residual del archivo `main.c` (no se actualizo al paralelizar); el binario usado en esta corrida es la version paralela con OpenMP habilitado (20 hilos).

Comando usado

```
1 timeout 30s ./ss -n <N> -f <F> -s 42 2>/dev/null \  
2 | grep "FPS=" | tail -n +6 \  
3 | awk -F'= ' '{s+=$2; c++} END {printf "FPS promedio: %.2f\n", s/c}'
```

El resto de las mediciones completas (las 15 filas de la bateria estandar por cada paso, el barrido de schedule y las curvas de escalabilidad) estan documentadas en `docs/Roadmap.md` dentro del repositorio del proyecto, junto con el historial de commits que muestra cuando se corrio cada bateria.